ES-CS301 — Data Structure & Algorithms

Semester III • 3L:0T:4P • 3 Credits

Module 1 Exam Notes: Introduction, Algorithm Analysis and Searching

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1 Elementary Data Organizations and Terminologies 2
1.1 Basic Data Organization Terminologies 2
1.2 Taxonomy of Data Structures 2
1.3 Fundamental Data Structure Operations 3
1.4 Array Memory Representations and Address Calculation 3
2 Algorithm Analysis and Performance Metrics 4
2.1 Performance Metrics: Time and Space Complexity 5
2.2 Frequency Count Method 5
3 Asymptotic Notations and Growth of Functions 5
3.1 Little-o and Little-omega Notations 6
3.2 Hierarchy and Properties of Growth Rates 6
3.3 The Time-Space Trade-Off 7
4 Searching Techniques and Algorithmic Complexity 7
4.1 Linear (Sequential) Search 7
4.2 Binary Search Technique 8
4.3 Comparison: Linear Search vs Binary Search 9

1. Elementary Data Organizations and Terminologies

Definition: Data Structure and Abstract Data Type (ADT)

A **data structure** is a specialized mathematical or logical organization of data items alongside a set of well-defined operations, designed to facilitate efficient storage, retrieval, and algorithmic modification in computer memory.

An **Abstract Data Type (ADT)** is an axiomatic mathematical specification of a data structure that defines:
1. The domain of data elements stored, and
2. The legal operations performed upon them together with their formal behavioral contracts,

independent of any underlying hardware architecture, compiler implementation, or memory layout.

1.1 Basic Data Organization Terminologies

Data items are structured hierarchically in memory from physical bits to logical models:

- **Data Item:** A single unit of value (e.g., an integer or a character).
- **Elementary vs Group Data Item:** An elementary item cannot be decomposed further (e.g., student ID). A group item can be divided into sub-items (e.g., full name divided into first, middle, and last names).
- **Entity and Attribute:** An entity represents a concrete real-world object (e.g., an employee). An attribute is an observable property or characteristic of that entity (e.g., age, salary).
- **Record:** A composite collection of heterogeneous data attributes representing a single entity.
- **File:** A collection of structurally identical records representing an entity set.

1.2 Taxonomy of Data Structures

Data structures are classified across multiple functional dimensions:

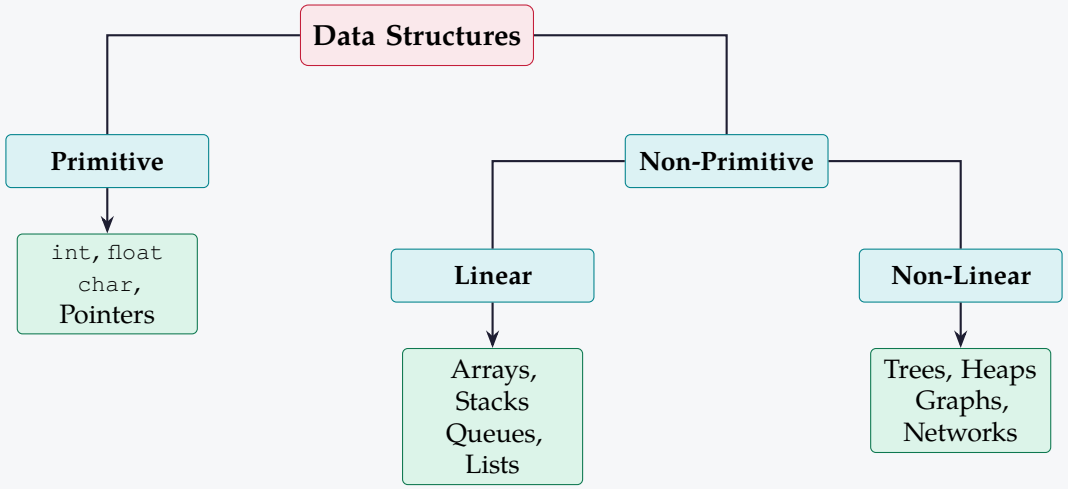
1. **Primitive vs Non-Primitive:** Primitive types are directly manipulated by underlying machine instructions (e.g., `int`, `float`, `char`, pointers). Non-primitive types are derived composite structures (e.g., arrays, lists, trees).
2. **Linear vs Non-Linear:** In a linear data structure, elements form a strictly sequential 1-to-1 ordering where every element (except first and last) has a unique predecessor and successor (e.g., Array, Linked List, Stack, Queue). In non-linear data structures, elements exhibit hierarchical (1-to-many) or interconnected (many-to-many) relationships (e.g., Trees, Graphs).
3. **Static vs Dynamic:** Static structures maintain a fixed memory allocation determined at compile time (e.g., static arrays). Dynamic structures allocate and deallocate physical memory blocks at runtime via system heaps (e.g., Linked Lists, dynamic BSTs).

© Biraj

4. **Homogeneous vs Heterogeneous:** Homogeneous structures enforce identical data types for all elements (e.g., arrays). Heterogeneous structures permit dissimilar types (e.g., C `struct`, records).

5. **Persistent vs Ephemeral:** Ephemeral structures destroy earlier versions upon modification; persistent structures preserve previous historical states after updates.

Structural Hierarchy and Taxonomy of Data Structures



1.3 Fundamental Data Structure Operations

Every computational structure supports standard operational primitives:

1. **Traversing:** Accessing each stored element exactly once to process or display data.
2. **Insertion:** Adding a new data element at a designated index, boundary, or ordered position.
3. **Deletion:** Removing an existing data item and releasing or compacting memory.
4. **Searching:** Finding the memory address or index of an element satisfying a target search key.
5. **Sorting:** Permuting elements into a monotonic ascending or descending collation order.
6. **Merging:** Combining two distinct ordered datasets into a unified ordered sequence.

1.4 Array Memory Representations and Address Calculation

An array is a homogeneous, contiguous collection of items accessed via integer offsets.

Let:

- $B =$ Base Address (memory address of the first byte of the array),
- $W =$ Size of each element in bytes (word size),
- $LB =$ Lower Bound of indices (default is 0 in C, 1 in mathematical texts).

Address Calculation Formulas for Multi-Dimensional Arrays

1. One-Dimensional Array $A[LB \dots UB]$:

$$\text{Loc}(A[i]) = B + W \cdot (i - LB) \tag{1}$$

© Biraj

2. Two-Dimensional Array $A[LB_1 \ldots UB_1,\ LB_2 \ldots UB_2]$**:**
Let $N_1 = UB_1 - LB_1 + 1$ (total rows) and $N_2 = UB_2 - LB_2 + 1$ (total columns).

- **Row-Major Order (RMO)** (consecutive row elements stored adjacently):

$$\text{Loc}(A[i][j]) = B + W \cdot \Big[(i - LB_1) \cdot N_2 + (j - LB_2)\Big] \tag{2}$$

- **Column-Major Order (CMO)** (consecutive column elements stored adjacently):

$$\text{Loc}(A[i][j]) = B + W \cdot \Big[(j - LB_2) \cdot N_1 + (i - LB_1)\Big] \tag{3}$$

Exam Numerical: 2D Array Address Calculation

Problem: An array $A[-5 \ldots 15,\ 10 \ldots 40]$ is allocated in byte-addressed RAM starting at base address $B = 1024$. Each element occupies $W = 4$ bytes. Determine the memory location of element $A[3][25]$ under both Row-Major and Column-Major storage schemes.

Solution:

- $LB_1 = -5,\ UB_1 = 15 \implies N_1 = 15 - (-5) + 1 = 21$ rows.
- $LB_2 = 10,\ UB_2 = 40 \implies N_2 = 40 - 10 + 1 = 31$ columns.
- Row offset: $i - LB_1 = 3 - (-5) = 8$.
- Column offset: $j - LB_2 = 25 - 10 = 15$.

1. Under Row-Major Order (RMO):

$$\begin{aligned}\text{Loc}(A[3][25]) &= 1024 + 4 \cdot [8 \cdot 31 + 15] = 1024 + 4 \cdot [248 + 15] \\ &= 1024 + 4 \cdot 263 = 1024 + 1052 = \mathbf{2076}\end{aligned}$$

2. Under Column-Major Order (CMO):

$$\begin{aligned}\text{Loc}(A[3][25]) &= 1024 + 4 \cdot [15 \cdot 21 + 8] = 1024 + 4 \cdot [315 + 8] \\ &= 1024 + 4 \cdot 323 = 1024 + 1292 = \mathbf{2316}\end{aligned}$$

2. Algorithm Analysis and Performance Metrics

Definition: Algorithm Criteria and Formal Characteristics

An **algorithm** is a finite, unambiguous, step-by-step computational procedure that accepts an initial input set and transforms it into a deterministic output set. Formally, every valid algorithm must satisfy:

1. **Input:** Generates output from zero or more well-defined external inputs.
2. **Output:** Produces at least one quantifiable mathematical or logical result.
3. **Definiteness:** Every operational step is completely clear, rigorous, and unambiguous.

© Biraj

4. **Finiteness:** Terminates strictly after a finite number of executable instructions for all valid inputs.
5. **Effectiveness:** Every instruction is basic enough to be executed exactly in finite time (feasible on physical hardware).

2.1 Performance Metrics: Time and Space Complexity

Algorithmic efficiency is measured independently of machine clock speed, compiler optimizations, or operating system jitter:

- **Space Complexity $S(P)$:** Total memory consumed by program P over its execution lifecycle:
$$S(P) = c + S_{\text{variable}}(I) \tag{4}$$
where c is the fixed part (instruction space, constants, scalar variables) and $S_{\text{variable}}(I)$ is the instance-dependent variable part (dynamically allocated memory, recursion stack activation frames).
- **Time Complexity $T(P)$:** Total computational operations or instruction steps executed as a mathematical function of input problem size n.

2.2 Frequency Count Method

The frequency count determines how many times each statement executes:

Step-Count Analysis of Nested Loops

Consider the matrix multiplication algorithm for two $n \times n$ matrices:

```
for (i = 0; i < n; i++)                    // Executes n + 1 times
    for (j = 0; j < n; j++)                // Executes n * (n + 1) times
    {
        C[i][j] = 0;                       // Executes n * n times
        for (k = 0; k < n; k++)           // Executes n * n * (n + 1) times
            C[i][j] += A[i][k]*B[k][j]; // Executes n * n * n times
    }
```

Total inner statement execution count: $f(n) = n^3$. As $n \to \infty$, the cubic term dominates all lower-order terms, yielding a time complexity of $\Theta(n^3)$.

3. Asymptotic Notations and Growth of Functions

Formal Definitions of Asymptotic Bounds

Let $f(n)$ and $g(n)$ be asymptotically non-negative functions defined on the set of natural numbers $\mathbb{N}$.

1. **Big-O Notation ($\mathcal{O}$ — Asymptotic Upper Bound):**
$$\mathcal{O}(g(n)) = \{f(n) : \exists\, c > 0,\ n_0 \in \mathbb{N} \text{ such that } 0 \leq f(n) \leq c \cdot g(n)\ \ \forall\, n \geq n_0\} \tag{5}$$

© Biraj

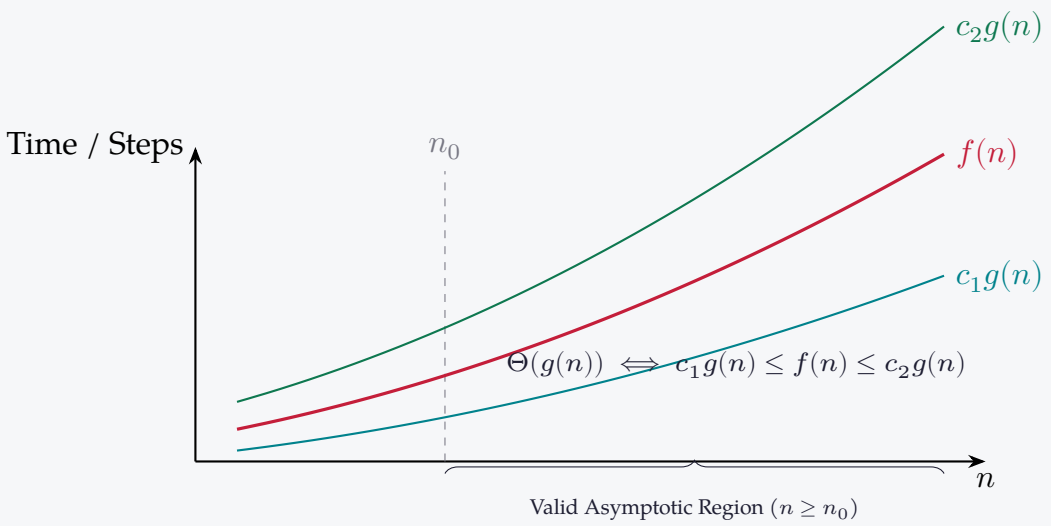
2. **Big-Omega Notation (Ω — Asymptotic Lower Bound):**

$$\Omega(g(n)) = \{f(n) : \exists\, c > 0,\ n_0 \in \mathbb{N} \text{ such that } 0 \le c \cdot g(n) \le f(n)\ \ \forall\, n \ge n_0\} \tag{6}$$

3. **Big-Theta Notation (Θ — Asymptotically Tight Bound):**

$$\Theta(g(n)) = \{f(n) : \exists\, c_1, c_2 > 0,\ n_0 \in \mathbb{N} \text{ s.t. } 0 \le c_1 g(n) \le f(n) \le c_2 g(n)\ \ \forall\, n \ge n_0\} \tag{7}$$

Geometric Interpretation of Asymptotic Notations



3.1 Little-o and Little-omega Notations

- **Little-o Notation (o — Strict Upper Bound):**

$$f(n) = o(g(n)) \iff \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \tag{8}$$

  Unlike $\mathcal{O}$, $f(n)$ cannot be asymptotically equal to $g(n)$ (e.g., $2n = o(n^2)$, but $2n \neq o(n)$).

- **Little-omega Notation (ω — Strict Lower Bound):**

$$f(n) = \omega(g(n)) \iff \lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty \tag{9}$$

3.2 Hierarchy and Properties of Growth Rates

Standard asymptotic complexities scale monotonically:

$$\begin{aligned} &\mathcal{O}(1) < \mathcal{O}(\log\log n) < \mathcal{O}(\log n) < \mathcal{O}(\sqrt{n}) < \mathcal{O}(n) \\ &\quad < \mathcal{O}(n\log n) < \mathcal{O}(n^2) < \mathcal{O}(n^3) < \mathcal{O}(2^n) < \mathcal{O}(n!) < \mathcal{O}(n^n) \end{aligned} \tag{10}$$

Fundamental Algebraic Properties:

- **Transitivity:** If $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$, then $f(n) = \Theta(h(n))$.
- **Symmetry:** $f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$.

© Biraj

- **Transpose Symmetry:** $f(n) = \mathcal{O}(g(n)) \iff g(n) = \Omega(f(n))$.
- **Theorem:** $f(n) = \Theta(g(n)) \iff f(n) = \mathcal{O}(g(n))$ and $f(n) = \Omega(g(n))$.

3.3 The Time-Space Trade-Off

In computational engineering, the **Time-Space Trade-off** dictates that algorithmic runtime can often be reduced at the expense of higher auxiliary memory usage, and vice-versa.

- **Memory-Heavy, Fast Execution:** Hash tables provide $\mathcal{O}(1)$ average lookup by utilizing extra array buckets. Pre-computed lookup tables or memoization arrays in Dynamic Programming trade space to eliminate redundant exponential recursive calls.
- **Memory-Light, Slow Execution:** In-place algorithms with $\mathcal{O}(1)$ auxiliary space (e.g., linear scanning or recomputing values on demand) save RAM at the cost of $\mathcal{O}(n)$ or $\mathcal{O}(n^2)$ processing delays.

4. Searching Techniques and Algorithmic Complexity

4.1 Linear (Sequential) Search

Linear search scans an unsorted array sequentially from index 0 to $n-1$, checking each element against target key K.

Linear Search Algorithm and Complexity Proof

Algorithm LinearSearch(A, n, K)**:**
1. For $i = 0$ to $n-1$ do:
2. If $A[i] == K$, return index i.
3. Return -1 (Search Key Not Found).

Complexity Derivation:

- **Best-Case Time Complexity:** Target key is at the first index ($A[0] == K$). Requires 1 comparison:
$$T_{\text{best}}(n) = \mathcal{O}(1) \tag{11}$$
- **Worst-Case Time Complexity:** Target key is at the last index ($A[n-1]$) or absent from array. Requires n comparisons:
$$T_{\text{worst}}(n) = \mathcal{O}(n) \tag{12}$$
- **Average-Case Time Complexity:** Assuming uniform distribution where key is present with probability p, and equally likely at any position ($1/n$):
$$T_{\text{avg}}(n) = \sum_{i=1}^{n} i \cdot P(\text{found at } i) = \sum_{i=1}^{n} i \cdot \frac{1}{n} = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2} \implies \mathcal{O}(n) \tag{13}$$
- **Auxiliary Space:** $\mathcal{O}(1)$ (in-place search).

© Biraj

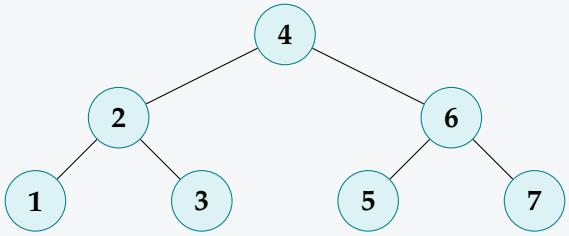
4.2 Binary Search Technique

Binary search operates on a **monotonically sorted** contiguous sequence via divide-and-conquer, halving the search space in each iteration.

Binary Search Algorithm (Iterative Implementation)

Algorithm BinarySearch(A, n, K)**:**
1. Set low $= 0$, high $= n - 1$.
2. While low $\leq$ high do:
3. Compute mid $=$ low $+ \lfloor(\text{high} - \text{low})/2\rfloor$. (*Prevents integer overflow*)
4. If $A[\text{mid}] == K$, return mid (Success).
5. Else if $A[\text{mid}] < K$, set low $=$ mid $+ 1$.
6. Else set high $=$ mid $- 1$.
7. Return -1 (Key Absent).

Decision Tree Representation of Binary Search on $n = 7$ Elements



At each stage, one comparison reduces the remaining search tree depth by 1. Total depth is $\lfloor \log_2 n \rfloor + 1$.

Binary Search Recurrence Relation and Complexity Derivation

In each step, the algorithm performs $\mathcal{O}(1)$ comparisons and reduces problem size n to $n/2$:

$$T(n) = T\left(\frac{n}{2}\right) + c, \qquad T(1) = 1 \tag{14}$$

Solving via Repeated Substitution (Unfolding):

$$\begin{aligned} T(n) &= T\left(\frac{n}{2}\right) + c = \left[T\left(\frac{n}{4}\right) + c\right] + c = T\left(\frac{n}{2^2}\right) + 2c \\ &= T\left(\frac{n}{2^k}\right) + k \cdot c \end{aligned}$$

The process terminates when the subproblem reaches size 1: $\frac{n}{2^k} = 1 \implies 2^k = n \implies k = \log_2 n$.

$$T(n) = T(1) + c \cdot \log_2 n = 1 + c \log_2 n \implies \square(\log_2 \mathbf{n}) \tag{15}$$

Summary of Bounds:
- **Best-Case:** $A[\text{mid}] == K$ on first comparison $\implies \mathcal{O}(\mathbf{1})$.
- **Worst-Case:** Element is at the leaf of the decision tree or absent $\implies \mathcal{O}(\log_2 \mathbf{n})$.
- **Average-Case:** Successful search across all decision tree nodes $\implies \mathcal{O}(\log_2 \mathbf{n})$.
- **Space Complexity:** $\mathcal{O}(1)$ for iterative version; $\mathcal{O}(\log_2 n)$ stack frames for recursive version.

© Biraj

4.3 Comparison: Linear Search vs Binary Search

Criterion	Linear Search	Binary Search
Prerequisite	Elements can be arbitrary/unsorted	Sequence **must** be monotonically sorted
Access Requirement	Sequential or Random access (Array or Linked List)	Random access required (Arrays only; inefficient on linked lists)
Best-Case Time	$\mathcal{O}(1)$ (first item matches)	$\mathcal{O}(1)$ (middle item matches)
Worst-Case Time	$\mathcal{O}(n)$	$\mathcal{O}(\log_2 n)$
Average-Case Time	$\mathcal{O}(n)$	$\mathcal{O}(\log_2 n)$
Space Complexity	$\mathcal{O}(1)$ in-place	$\mathcal{O}(1)$ iterative, $\mathcal{O}(\log n)$ recursive
Hardware Efficiency	Excellent CPU cache spatial locality	Multiple cache misses on large data jumps
Algorithm Paradigm	Brute-force linear scan	Divide-and-Conquer

© Biraj

Quick Revision — Module 1

Introduction, Algorithm Analysis and Searching Techniques | ES-CS301

Key Formulas and Asymptotic Relations at a Glance

1D Array Address	$\text{Loc}(A[i]) = B + W \cdot (i - LB)$
2D Row-Major Address	$\text{Loc}(A[i][j]) = B + W \cdot [(i - LB_1) \cdot N_2 + (j - LB_2)]$
2D Column-Major Address	$\text{Loc}(A[i][j]) = B + W \cdot [(j - LB_2) \cdot N_1 + (i - LB_1)]$
Asymptotic Bounds $(\mathcal{O}, \Omega, \Theta)$	$f(n) \leq c_1 g(n)$ $(\mathcal{O})$, $c_2 g(n) \leq f(n)$ (Ω), $c_3 g(n) \leq f(n) \leq c_4 g(n)$ (Θ)
Strict Limits (o, ω)	$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$ (o), $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$ (ω)
Binary Search Recurrence	$T(n) = T(n/2) + \Theta(1) \implies T(n) = \Theta(\log_2 n)$
Linear Search Bounds	Best-Case: $\mathcal{O}(1)$, Worst-Case / Average: $\mathcal{O}(n)$
Complexity Hierarchy	$\mathcal{O}(1) < \mathcal{O}(\log n) < \mathcal{O}(n) < \mathcal{O}(n \log n) < \mathcal{O}(n^2) < \mathcal{O}(2^n) < \mathcal{O}(n!)$

One-Line Definitions

- **Data Structure:** A logical and mathematical organization of data elements along with designated legal operations.
- **Abstract Data Type (ADT):** A mathematical model for data types defining behavior from user perspective without implementation details.
- **Linear Data Structure:** A data structure where elements form an ordered sequential sequence with 1-to-1 adjacent relations.
- **Row-Major Order (RMO):** A 2D mapping layout where all elements of row i precede row $i + 1$ in contiguous physical memory.
- **Column-Major Order (CMO):** A 2D mapping layout where all elements of column j precede column $j + 1$ in contiguous physical memory.
- **Big-O ($\mathcal{O}$):** An asymptotic notation defining a formal mathematical upper bound on algorithm computational growth.
- **Time-Space Trade-off:** The engineering principle where computational runtime is decreased by allocating extra memory.
- **Binary Search:** A divide-and-conquer algorithm on sorted arrays halving search space in $\mathcal{O}(\log_2 n)$ comparisons.

© Biraj

Frequently Asked / Viva Questions (Part 1)

Q1. What is the difference between a Data Type and a Data Structure?

A data type is a primitive classification defining a set of values and allowable low-level hardware operations (e.g., `int`, `float`). A data structure is a higher-level collection of data elements organized logically with specific algorithms to store and manipulate them efficiently in memory (e.g., arrays, stacks, trees).

Q2. Why do we prefer asymptotic notations over clock time for analyzing algorithms?

Physical clock time depends on machine architecture, processor speed, compiler optimizations, background processes, and cache memory. Asymptotic notations evaluate performance purely as a function of input size n, independent of hardware, providing an objective, universal measure of scalability.

Q3. State the necessary and sufficient condition for $f(n) = \Theta(g(n))$.

The function $f(n) = \Theta(g(n))$ if and only if $f(n) = \mathcal{O}(g(n))$ and $f(n) = \Omega(g(n))$. That is, $g(n)$ serves simultaneously as both the asymptotic upper bound and lower bound within positive constant multipliers c_1 and c_2.

Q4. Why must an array be sorted prior to executing Binary Search?

Binary search relies on the monotonicity of elements. When the target key K is compared with the middle element $A[\text{mid}]$, the algorithm assumes that all elements to the left are $\leq A[\text{mid}]$ and all elements to the right are $\geq A[\text{mid}]$. Without sorted order, discarding half the array could eliminate the target key.

Q5. Why is Binary Search inefficient when applied to a Singly Linked List?

Binary search requires finding the midpoint in $\mathcal{O}(1)$ time. In a linked list, random access is not supported ($A[i]$ takes $\mathcal{O}(i)$ traversal steps). Locating the middle node takes $\mathcal{O}(n)$ time, which negates the divide-and-conquer speedup, resulting in overall $\mathcal{O}(n)$ complexity.

Q6. Explain the concept of integer overflow in computing the midpoint in Binary Search and how it is resolved.

Writing `mid = (low + high) / 2` can cause integer overflow if the sum `low + high` exceeds the maximum 32-bit signed integer value ($2^{31} - 1$). The overflow wraps to a negative number, causing an index out-of-bounds fault. It is avoided using `mid = low + (high - low) / 2`.

Q7. Differentiate between Big-O ($\mathcal{O}$) and Little-o (o).

Big-O establishes an asymptotic upper bound that may be tight ($2n^2 = \mathcal{O}(n^2)$ is valid). Little-o establishes a strict, non-tight upper bound such that $f(n)$ grows strictly slower than $g(n)$ as $n \to \infty$ ($\lim_{n\to\infty} f(n)/g(n) = 0$). Thus $2n = o(n^2)$, but $2n^2 \neq o(n^2)$.

© Biraj

Frequently Asked / Viva Questions (Part 2)

Q8. What is the difference between Space Complexity and Auxiliary Space?

Space complexity is the total memory used by an algorithm, including both the input data and any extra working memory. Auxiliary space refers exclusively to the extra temporary or scratchpad memory allocated by the algorithm beyond the input size.

Q9. Provide an example of an engineering Time-Space trade-off.

In calculating Fibonacci numbers: naive recursion takes $\mathcal{O}(2^n)$ time and $\mathcal{O}(n)$ call-stack space. Using dynamic programming memoization with an auxiliary array of size n, runtime drops dramatically to $\mathcal{O}(n)$ time at the expense of $\mathcal{O}(n)$ extra storage space.

Q10. What are the worst-case, best-case, and average-case scenarios for Linear Search?

Best case is $\mathcal{O}(1)$ (element found at index 0). Worst case is $\mathcal{O}(n)$ (element is at index $n-1$ or absent). Average case is $\mathcal{O}(n)$ (specifically $(n+1)/2$ comparisons under a uniform random distribution).

Summary Matrix: Comprehensive Comparison of Searching and Analysis Fundamentals

Topic / Technique	Core Governing Relation	Complexity Bound	Key Engineering Takeaway
Row-Major 2D Addressing	**Loc** $= B + W[(i - LB_1)N_2 + (j - LB_2)]$	$\mathcal{O}(1)$ **compute time**	C/C++ memory layout; row iteration preserves cache locality.
Column-Major 2D Addressing	**Loc** $= B + W[(j - LB_2)N_1 + (i - LB_1)]$	$\mathcal{O}(1)$ **compute time**	Fortran/MATLAB layout; column-by-column iteration preferred.
Big-O Notation ($\mathcal{O}$)	$f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$	**Upper bound**	Worst-case computational ceiling guarantees.
Big-Theta Notation (Θ)	$c_1 g(n) \leq f(n) \leq c_2 g(n)$	**Tight bound**	Exact algorithmic rate of growth.
Linear Search	**Sequential traversal** $A[0 \ldots n-1]$	**Worst:** $\mathcal{O}(n)$, **Best:** $\mathcal{O}(1)$	Unsorted or small lists; works natively on linked lists.
Binary Search	$T(n) = T(n/2) + \mathcal{O}(1)$	**Worst:** $\mathcal{O}(\log n)$, **Best:** $\mathcal{O}(1)$	Strictly requires sorted array and random access memory.

© Biraj